先进计算与数字工程研究中心

# PATTERN RECOGNITION AND MACHINE LEARNING

## Lecture 17 – Deep Learning III: Training Practice and Applications

**Chizhi Chris ZHANG**

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

# Three periods, three training questions

| Period | What we learn | What you should be able to do |
|---|---|---|
| 1 | Mini-batches, momentum, Adam and schedules | Calculate an update; explain why step size changes |
| 2 | Initialization, normalization, residual paths and regularization | Identify what happens during training and inference |
| 3 | Learning curves, augmentation and transfer learning | Diagnose a problem and choose a justified next step |

### One running task

Train an image classifier with limited labeled data. We will connect each training method to this task rather than treat methods as isolated recipes.

# One training step has five linked operations

1. Draw a mini-batch $(x_i, y_i)$ and compute predictions $f_\theta(x_i)$.
2. Average the sample losses: $L_B = \frac{1}{B} \sum_i \ell(f_\theta(x_i), y_i)$.
3. Backpropagate to obtain $g_t = \nabla_\theta L_B$.
4. Let the optimizer combine $g_t$ with its stored state and update $\theta$.
5. Repeat with new batches; evaluate separately on fixed validation data.

**For our classifier**

The output layer produces logits, cross-entropy measures label error, and gradients tell every trainable layer how to change.

**A common error**

Old gradients accumulate unless they are cleared intentionally.

# Loss reduction changes the effective step

For sample losses $\ell_1, \ldots, \ell_B$, distinguish

$$L_{\text{mean}} = B^{-1} \sum_i \ell_i, \qquad L_{\text{sum}} = \sum_i \ell_i.$$

| Reduction | Gradients $(2, -1, 3, 0)$ | Step at $\eta = 0.1$ |
|---|---|---|
| Mean | $g = 1$ | $\Delta\theta = -0.1$ |
| Sum | $g = 4$ | $\Delta\theta = -0.4$ |

- The optimizer cannot infer which reduction was intended.
- Changing batch size under a sum changes gradient scale even if examples have similar gradients.
- Read the loss definition before reusing a learning rate from another experiment.

# A mini-batch trades exactness for frequent updates

$$\nabla L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\theta), \qquad g_B = \frac{1}{B}\sum_{i \in \mathcal{B}} \nabla \ell_i(\theta).$$

- Full-batch gradient uses all $N$ cases: accurate for the empirical loss, expensive per update.
- A random mini-batch estimates the full gradient. Uniform sampling makes it unbiased; an individual estimate is still noisy.
- Larger $B$ usually reduces sampling noise but uses more memory and yields fewer updates per epoch.
- Shuffling changes which cases contribute to successive updates; repeated video frames are not independent new examples.

### Compare fairly

Keep processed samples and training time visible when changing batch size.

# Count updates before comparing batch sizes

Suppose $N = 1000$ training images and keep the last incomplete batch.

| Batch size | Updates per epoch | Updates in three epochs |
|---|---|---|
| 32 | $\lceil 1000/32 \rceil = 32$ | 96 |
| 128 | $\lceil 1000/128 \rceil = 8$ | 24 |

- Both runs process 3000 examples, but they make different numbers of parameter updates.
- A GPU can process a larger batch efficiently, yet memory and optimization behavior can change.
- An update-based schedule also advances differently unless its budget is adjusted.

## Comparison

Record processed examples, update count and elapsed time together with validation performance.

# SGD: calculate the gradient before the step

For a scalar parameter, four sample gradients are $2, -1, 3, 0$. Their mean is

$$g_B = (2 - 1 + 3 + 0)/4 = 1.$$

Starting from $\theta_t = 5$ with $\eta = 0.1$ gives $\theta_{t+1} = 5 - 0.1 \times 1 = 4.9$.

| Selected batch | Mean gradient | Next parameter |
|---|---|---|
| Samples 1 and 2 | 0.5 | 4.95 |
| Samples 3 and 4 | 1.5 | 4.85 |
| All four samples | 1.0 | 4.90 |

### Interpretation

Different batches produce different steps. Averaging loss rather than summing it keeps the learning-rate meaning less sensitive to batch size.

# Derive a batch gradient from a tiny linear model

Use $\hat{y} = wx$, $\ell = \frac{1}{2}(wx - y)^2$ and two examples $(1, 2), (2, 4)$.

$$\frac{\partial \ell}{\partial w} = (wx - y)x.$$

At $w = 1$:

| Example | Residual $wx - y$ | Gradient |
|---|---|---|
| $(1, 2)$ | $-1$ | $-1 \times 1 = -1$ |
| $(2, 4)$ | $-2$ | $-2 \times 2 = -4$ |

The mean is $g = -2.5$. With $\eta = 0.1$, $w_{\text{new}} = 1 - 0.1(-2.5) = 1.25$.

### Interpretation

Predictions are too small. The negative gradient increases $w$, moving toward the shared relation $y = 2x$.

# Exercise: calculate an update and explain its sign

A scalar parameter starts at $\theta = 2$. A batch gives gradients $(4, 0, -2, 2)$, and $\eta = 0.05$.

1. Compute the mean gradient and the next parameter.
2. Recompute the step if the loss reduction is a sum.
3. Explain why one negative sample gradient does not force the batch update to increase the parameter.

| Object | Meaning | Keep separate |
|---|---|---|
| Sample gradients | Four local opinions | Not yet one batch direction |
| Batch reduction | Combines those opinions | Mean or sum must be stated |
| Learning rate | Scales the selected direction | Does not choose the sign |

# Answer: the batch direction follows the combined gradient

$$g_{\text{mean}} = (4 + 0 - 2 + 2)/4 = 1, \qquad \theta_{\text{new}} = 2 - 0.05(1) = 1.95.$$

| Reduction | Gradient | Parameter after update |
|---|---|---|
| Mean | 1 | 1.95 |
| Sum | 4 | $2 - 0.05(4) = 1.80$ |

- The negative sample contribution partly cancels the positive ones, but the total remains positive.
- Subtracting a positive gradient decreases the parameter.
- Averaging helps avoid a purely batch-count-driven increase in step scale. Actual sample composition can still change the direction.

# Learning rate controls how far, not which way

Use $L(w) = \frac{1}{2}w^2$, so $\nabla L = w$ and $w_{t+1} = (1 - \eta)w_t$.

| $\eta$ | Start at $w_0 = 2$ | Behavior | Lesson |
|---|---|---|---|
| 0.1 | $2, 1.8, 1.62, \ldots$ | Slow contraction | Safe can still be slow |
| 1.5 | $2, -1, 0.5, \ldots$ | Alternating, shrinking | Overshoot can converge |
| 2.1 | $2, -2.2, 2.42, \ldots$ | Alternating, growing | Too large diverges |

**For this quadratic**

Convergence requires $0 < \eta < 2$. A neural network has many local curvatures, so this interval is not a universal recommendation.

**Numerical failure**

NaN can also come from invalid data or unstable operations; inspect the first failing quantity.

# Successive values distinguish shrinking from growing oscillations

For $L(w) = \frac{1}{2}w^2$, compare two rates from $w_0 = 2$.

| Rate | First three values | First three losses |
|---|---|---|
| 0.1 | $2, 1.8, 1.62$ | $2, 1.62, 1.3122$ |
| 2.1 | $2, -2.2, 2.42$ | $2, 2.42, 2.9282$ |

- A changing sign alone does not diagnose divergence; compare magnitudes and losses over successive steps.
- With $\eta = 1.5$, values $2, -1, 0.5$ alternate while their magnitudes shrink.
- A neural-network curve is noisier, so inspect a sequence of updates rather than one increase.

# A narrow valley makes plain SGD zigzag

For $L(w_1, w_2) = \frac{1}{2}(w_1^2 + 100w_2^2)$, the gradient is $(w_1, 100w_2)$.

- The steep direction $w_2$ requires a small step for stability.
- The same small learning rate makes progress in the flat direction $w_1$ slow.
- Mini-batch noise adds local changes in direction; reducing the rate does not remove the geometric mismatch.

**At $(1, 1)$ with $\eta = 0.01$**

The next point is $(0.99, 0)$. With $\eta = 0.03$, it is $(0.97, -2)$: the steep coordinate overshoots.

**Two possible improvements**

Accumulate consistent directions with momentum, or rescale coordinates using gradient history.

# A coordinate scale can dominate an otherwise useful direction

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

At $(w_1, w_2) = (1, 1)$ for $L = \frac{1}{2}(w_1^2 + 100w_2^2)$, $g = (1, 100)$.

| Update design | Direction before learning rate | Change at $\eta = 0.01$ |
|---|---|---|
| Ordinary gradient | $(1, 100)$ | $(-0.01, -1)$ |
| Illustrative rescaling $D$ | $Dg = (1, 1)$ | $(-0.01, -0.01)$ |

Here $D = \mathrm{diag}(1, 0.01)$ is chosen for this demonstration, not learned by this table.

### Meaning

Rescaling changes relative coordinate progress. Momentum and adaptive optimizers exploit different historical information; neither directly knows the best scaling for every problem.

# Momentum remembers useful directions

Use a gradient accumulator $m_0 = 0$:

$$m_t = \beta m_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta m_t.$$

- $g_t$: current mini-batch gradient; $m_t$: accumulated direction; $0 \le \beta < 1$: memory strength.
- Repeated gradients with the same sign reinforce one another.
- Opposite gradients partly cancel, reducing zigzagging in a steep direction.
- Older gradients receive geometrically smaller weights: $m_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \cdots$.

**Conventions**

Some books include $(1 - \beta)$ in the accumulator or store velocity $v = -\eta m$. Check the definition before copying a learning rate.

# Expand three momentum steps to see the stored history

Using $m_t = \beta m_{t-1} + g_t$ with $m_0 = 0$,

$$m_3 = g_3 + \beta g_2 + \beta^2 g_1.$$

For $g_1 = g_2 = g_3 = 2$ and $\beta = 0.9$:

| Contribution | Weight | Contribution to $m_3$ |
|---|---|---|
| Current $g_3$ | 1 | 2 |
| Previous $g_2$ | 0.9 | 1.8 |
| Earlier $g_1$ | 0.81 | 1.62 |

Thus $m_3 = 5.42$; at $\eta = 0.1$, this update moves $-0.542$.

## Meaning

Old gradients fade geometrically. Consistent new gradients reinforce them; changes in direction can cancel them.

# Momentum: work through two different gradient histories

Let $\theta_0 = 5$, $\eta = 0.1$, $\beta = 0.9$, $m_0 = 0$.

| History | $m_1, \theta_1$ | $m_2$ | $\theta_2$ |
|---|---|---|---|
| $g_1 = 2, g_2 = 2$ | $2, 4.8$ | $0.9(2) + 2 = 3.8$ | 4.42 |
| $g_1 = 2, g_2 = -2$ | $2, 4.8$ | $0.9(2) - 2 = -0.2$ | 4.82 |

- Consistent direction: the second move grows from 0.2 to 0.38.
- Reversal: the second move is only $+0.02$, because history and the new gradient cancel.
- Without momentum, the corresponding second moves would be $-0.2$ and $+0.2$.

### Read the mechanism

Momentum can accelerate a stable direction and smooth an alternating one; it can also overshoot when an old direction becomes stale.

# A sustained reversal eventually replaces the old direction

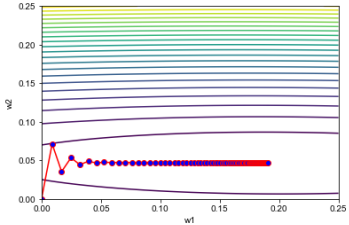
Start from $\theta_0 = 5$, $m_0 = 0$, $\beta = 0.9$, $\eta = 0.1$; use gradients $(2, -2, -2)$.

| Step | Accumulator | Parameter after step |
|---|---|---|
| 1 | $m_1 = 2$ | $\theta_1 = 4.8$ |
| 2 | $m_2 = 0.9(2) - 2 = -0.2$ | $\theta_2 = 4.82$ |
| 3 | $m_3 = 0.9(-0.2) - 2 = -2.18$ | $\theta_3 = 5.038$ |

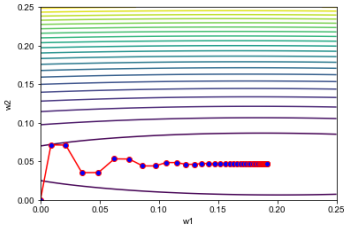
- One reversal is strongly smoothed; repeated reversals establish a new stored direction.
- A larger accumulator is not automatically a better step: the learning rate and actual loss still matter.
- The supplied sequence isolates the mechanism; it is not a claim about loss improvement in every step.

# Read the optimization paths in the reference figure

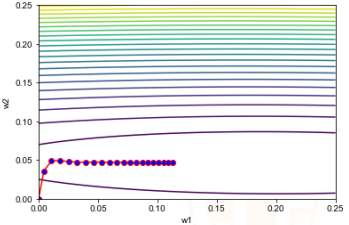
**Plain gradient descent**




**Momentum**




**Nesterov (extension)**




- Plain descent repeatedly crosses the steep direction. Momentum smooths crossings while advancing along the valley.
- In our arithmetic, gradients $2, -2$ partly cancel; $2, 2$ reinforce. Nesterov evaluates a look-ahead gradient.
- These original plots illustrate update geometry; one example does not rank optimizers universally.

# Compare mechanisms before comparing optimizer names

| Method | Information used | Mechanism to explain |
|---|---|---|
| Plain SGD | Current gradient | Follow the current estimated direction |
| Momentum | Current and past signed gradients | Reinforce consistent directions; smooth reversals |
| RMSProp | Squared-gradient moving average | Rescale coordinates from recent magnitudes |
| Adam | Signed and squared moving averages | Combine direction, scale and initial-bias correction |

- Tune a suitable rate for each method; equal numerical rates are not equal effective updates.
- Match data split and compute budget, then inspect validation and cost.

# Adaptive scaling gives each coordinate a different step

AdaGrad stores $s_t = s_{t-1} + g_t^2$; RMSProp replaces the sum with a moving average:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}.$$

All squares, roots and divisions act coordinate by coordinate.

- Repeatedly large gradients make $s_t$ large and reduce the effective step.
- AdaGrad's accumulated sum never decreases, so its step can become too small.
- RMSProp forgets distant history and can adapt to a changing gradient scale.

### Scale effect

With $g = (1, 10)$ and $\sqrt{s} = (1, 10)$, the scaled gradient is $(1, 1)$, before multiplication by $\eta$.

# RMSProp: a large first gradient changes the next scale

Use $s_t = 0.9s_{t-1} + 0.1g_t^2$, $s_0 = 0$, $\eta = 0.1$, with $g_1 = 2, g_2 = 0.5$.

| Step | Squared-gradient record | Update magnitude, ignoring tiny $\epsilon$ |
|---|---|---|
| 1 | $s_1 = 0.1(4) = 0.4$ | $0.1(2)/\sqrt{0.4} \approx 0.3162$ |
| 2 | $s_2 = 0.9(0.4) + 0.1(0.25) = 0.385$ | $0.1(0.5)/\sqrt{0.385} \approx 0.0806$ |

- The second denominator still reflects the earlier large gradient.
- The record stores magnitude, not sign; the current $g_t$ still supplies the direction.
- This is RMSProp without Adam-style bias correction.

# Adam combines direction, scale and bias correction

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\hat{m}_t = m_t/(1-\beta_1^t), \qquad \hat{v}_t = v_t/(1-\beta_2^t),$$
$$\theta_{t+1} = \theta_t - \eta \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon).$$

- $m_t$ averages signed gradients; $v_t$ averages squared gradients.
- Starting both at zero makes early estimates too small; the hats correct this initialization bias.
- $\beta_1 = 0.9, \beta_2 = 0.999$ are common starting values, not a guarantee of best performance.

**Still tune the rate**

Coordinate adaptation changes effective steps. It does not remove the need for a suitable global $\eta$.

# Adam: the first update is easy to check

Let $g_1 = 2$, $m_0 = v_0 = 0$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\eta = 0.1$.

| Quantity | Calculation | Result |
|---|---|---|
| $m_1$ | $0.1 \times 2$ | 0.2 |
| $v_1$ | $0.001 \times 4$ | 0.004 |
| $\hat{m}_1$ | $0.2/(1 - 0.9)$ | 2 |
| $\hat{v}_1$ | $0.004/(1 - 0.999)$ | 4 |
| Update | $-0.1(2)/(\sqrt{4} + \epsilon)$ | approximately $-0.1$ |

**Compare**

Plain SGD would move $-0.2$ with this same learning rate. Different optimizer definitions make equal numeric learning rates behave differently.

**Do not generalize the toy result**

Later updates depend on the full gradient history, and a near-zero denominator needs $\epsilon$.

# Adam: calculate the second update from both histories

Continue the previous Adam example with $g_1 = 2, g_2 = 0.5$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

| Quantity | Calculation | Result |
|---|---|---|
| $m_2$ | $0.9(0.2) + 0.1(0.5)$ | 0.23 |
| $v_2$ | $0.999(0.004) + 0.001(0.25)$ | 0.004246 |
| $\hat{m}_2$ | $0.23/(1 - 0.9^2)$ | 1.2105 |
| $\hat{v}_2$ | $0.004246/(1 - 0.999^2)$ | 2.1241 |

At $\eta = 0.1$, the second change is approximately $-0.1(1.2105)/\sqrt{2.1241} = -0.0831$.

### Meaning

Even though the current gradient is 0.5, the update depends on past direction and scale. It is not SGD's $-0.05$.

# Bias correction compensates for a zero initial record

For a constant gradient $g_t = 2$ and $\beta_1 = 0.9$,

$$m_t = 2(1 - 0.9^t), \qquad \hat{m}_t = m_t/(1 - 0.9^t) = 2.$$

| Step | Raw $m_t$ | Correction factor | Corrected value |
|---|---|---|---|
| 1 | 0.2 | 0.1 | 2 |
| 2 | 0.38 | 0.19 | 2 |
| 10 | approximately 1.3026 | approximately 0.6513 | 2 |

- The raw average gradually approaches the constant gradient.
- Early small values partly reflect the initial zero, not a genuinely small gradient.
- The same principle applies to the squared-gradient record.

# Schedules move from exploration to refinement

| Schedule | Rule | Why use it |
|---|---|---|
| Step decay | Multiply $\eta$ by a factor at chosen epochs | Simple controlled reductions |
| Cosine decay | $\eta_t = \frac{\eta_0}{2}[1 + \cos(\pi t/T)]$ | Smoothly approach smaller steps |
| Validation plateau | Reduce after sustained lack of improvement | Respond to a measured plateau |

- A large early rate can make rapid progress; a smaller late rate can refine the solution.
- Fix the total budget $T$ before interpreting a schedule.
- A schedule changes the global step scale. Adam still benefits from scheduling.

**Cosine example**

For $\eta_0 = 0.01$, rates at $t = 0, T/2, T$ are $0.01, 0.005, 0$.

# Step and cosine schedules spend the same budget differently

Set $T = 100$, initial rate 0.01. Step decay multiplies by 0.1 at updates 50 and 75; cosine uses $\eta_t = 0.005[1 + \cos(\pi t/100)]$.

| Update | Step-decay rate | Cosine rate |
|---|---|---|
| 0 | 0.01 | 0.01 |
| 50 | 0.001 | 0.005 |
| 100 | 0.0001 | 0 |

- Both start equally and end smaller, but intermediate progress can differ.
- Declare whether a boundary update uses the old or new rate.
- Compare the actual rate sequence and validation results; the schedule name alone does not determine performance.

# Warmup and optimizer choice need a controlled comparison

## Warmup

Start with a small rate and increase to the intended rate over the first $W$ updates: $\eta_t = \eta_{\max} t/W$ for $t \leq W$. Then apply the main schedule.

- Useful when early activations, gradients or optimizer moments are poorly scaled; it is not always necessary.
- Compare SGD with momentum and AdamW using separately tuned rates, the same split, similar compute and a fixed validation rule.
- Keep optimizer state when resuming training. A saved parameter tensor alone cannot reproduce momentum or Adam history.
- Report validation behavior and training cost; fast initial loss decrease is only one part of the result.

## A concrete warmup

With $W = 1000$ and $\eta_{\max} = 10^{-3}$, update 100 uses $10^{-4}$.

# Exercise: identify which part of training was changed

| Change | Primary object | Explain the mechanism |
|---|---|---|
| Add momentum at the same rate | Direction history | Reuse signed past gradients |
| Reduce the rate after a plateau | Global step scale | Smaller movement per update |
| Increase batch size | Gradient estimate and update count | Different noise, memory and frequency |
| Use early warmup | Early rate sequence | Build up the intended step gradually |

**Discuss**

Which change directly makes the current gradient estimate less noisy? Which stores a memory? Why should their effects not be described with one phrase such as “training is more stable”?

# Initialization determines the first signal scale

- If a layer sends almost zero activations onward, later layers have little signal to use.
- If magnitudes grow across layers, activations can saturate or become numerically unstable.
- Identical incoming parameters and symmetric outgoing connections preserve symmetry: hidden units receive identical gradients and learn identical features.
- Random initialization breaks symmetry; its variance should respect the layer's input size and activation.

### Bias versus weight

Biases can commonly start at zero. Making every hidden-layer weight zero is different because it removes feature diversity.

### Goal

Keep forward activations and backward gradients at a usable scale rather than choose arbitrary small random numbers.

# Zero hidden parameters can preserve an unhelpful symmetry

Consider $\hat{y} = v_1 \tanh(w_1 x) + v_2 \tanh(w_2 x)$.

- If $w_1 = w_2$ and $v_1 = v_2$, both hidden units have symmetric roles, identical activations and identical gradient updates.
- If all four weights are zero, the hidden activations are zero; gradients to the output weights are zero, and zero output weights block gradients to incoming weights.
- Independent random incoming weights break this exact symmetry.
- Zero biases are compatible with distinct random weights; biases alone do not need the same treatment as every hidden weight.

### Condition

Identical incoming weights alone do not guarantee identical gradients when outgoing connections differ.

# Xavier and He scaling follow the layer size

For $z_j = \sum_{i=1}^{n_{\text{in}}} w_{ji} x_i$, independent zero-mean terms approximately give

$$\operatorname{Var}(z_j) \approx n_{\text{in}} \operatorname{Var}(w) \operatorname{Var}(x).$$

| Choice | Typical variance | Suitable starting point |
|---|---|---|
| Xavier | $2/(n_{\text{in}} + n_{\text{out}})$ | Approximately linear / tanh layers |
| He, fan-in | $2/n_{\text{in}}$ | ReLU layers |

**For $n_{\text{in}} = 100$**

He variance is 0.02, so standard deviation is $\sqrt{0.02} \approx 0.141$, not 0.02.

**Approximation**

The argument uses distributional assumptions. Normalization and trained weights change these distributions; initialization is a starting design.

# Calculate initialization scales for two widths

He fan-in initialization uses $\mathrm{Var}(w) = 2/n_{\mathrm{in}}$; a normal sampler commonly takes its square root as the standard deviation.

| Layer | Weight variance | Standard deviation |
|---|---|---|
| ReLU, $n_{\mathrm{in}} = 100$ | 0.02 | approximately 0.1414 |
| ReLU, $n_{\mathrm{in}} = 400$ | 0.005 | approximately 0.0707 |
| Xavier, $n_{\mathrm{in}} = 100, n_{\mathrm{out}} = 50$ | $2/150 \approx 0.0133$ | approximately 0.1155 |

### Meaning

More summed inputs call for smaller individual weights. The variance rule and sampler argument must refer to the same distributional quantity.

# Standardized inputs do not fix every hidden layer

- Input standardization uses statistics fitted on the training set and then applies the same transformation to new data.
- Hidden representations change as network weights change; their scales can differ substantially across layers.
- Normalization layers transform intermediate values using a defined set of axes.
- Learned scale and shift allow the network to retain useful representational flexibility after normalization.

### Our image classifier

Input pixel normalization is fixed preprocessing. BatchNorm inside the CNN acts on intermediate channel activations during training. These are related operations with different statistics.

### Meaning

Normalization does not make arbitrary activations exactly Gaussian and does not guarantee good generalization.

# Training statistics must define the preprocessing transform

Use training values $(0, 2)$, with population mean $\mu = 1$ and standard deviation $\sigma = 1$. A new value 4 becomes $(4 - 1)/1 = 3$.

| Statistics source | Mean / standard deviation | Transformed new value |
|---|---|---|
| Training only | $1, 1$ | 3 |
| Training plus new value | $2, \sqrt{8/3}$ | approximately 1.225 |

- These are different transforms, not two equivalent numerical implementations.
- Using validation/test values to fit preprocessing changes the intended evaluation boundary.
- BatchNorm later acts on intermediate activations with its own mode-dependent statistics.

# BatchNorm normalizes a feature across a mini-batch

For one feature with batch values $z_1, \ldots, z_B$,

$$\mu_B = \frac{1}{B}\sum_i z_i, \quad \sigma_B^2 = \frac{1}{B}\sum_i (z_i - \mu_B)^2, \quad y_i = \gamma \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta.$$

- $\mu_B, \sigma_B^2$ are computed statistics, not parameters learned by the optimizer.
- $\gamma, \beta$ are trainable feature-wise scale and shift.
- In a CNN, statistics commonly aggregate batch and spatial positions separately for each channel.
- Tiny or unrepresentative batches can produce unstable estimates.

### Keep the axes clear

BatchNorm compares the same feature across examples; it does not normalize all numbers in a batch indiscriminately.

# Normalization axes determine which values are compared

| Tensor and layer | Statistics group | Values per group |
|---|---|---|
| CNN: $B = 2, C = 3, H = W = 4$, BatchNorm | One channel over batch and space | $2 \times 4 \times 4 = 32$ |
| Tokens: $B = 2, T = 4, D = 6$, LayerNorm over $D$ | One token's features | 6 |

- CNN BatchNorm computes separate channel statistics; it does not combine all three channels into one mean.
- Token LayerNorm computes a separate feature mean for every token.
- Read the configured axes before assuming two normalization layers are interchangeable.

# BatchNorm: calculate normalized and learned outputs

For $z = (1, 3, 5, 7)$, ignoring a tiny $\epsilon$ for this arithmetic:

$$\mu_B = 4, \qquad \sigma_B^2 = (9 + 1 + 1 + 9)/4 = 5.$$

| | | | | |
|---|---|---|---|---|
| Input $z_i$ | 1 | 3 | 5 | 7 |
| Normalized $(z_i - 4)/\sqrt{5}$ | $-1.342$ | $-0.447$ | 0.447 | 1.342 |
| With $\gamma = 2, \beta = 1$ | $-1.683$ | 0.106 | 1.894 | 3.683 |

- The normalized batch has mean zero and variance approximately one.
- Learned scale and shift deliberately change those values afterward.
- Another batch has another mean and variance, so the same raw activation can have a different normalized value during training.

# One activation can receive two training normalizations

Compare the same activation $z = 3$ in two batches, ignoring tiny $\epsilon$.

| Batch values | Mean / variance | Normalized value of 3 |
|---|---|---|
| $(1, 3, 5, 7)$ | $4, 5$ | $-1/\sqrt{5} \approx -0.447$ |
| $(3, 5, 7, 9)$ | $6, 5$ | $-3/\sqrt{5} \approx -1.342$ |

- The raw activation did not change. Its position relative to batch partners changed.
- Learned $\gamma, \beta$ are then applied to the normalized value.
- This batch dependence is present in training mode; saved inference statistics define a different, fixed transform.

# Exercise: separate normalized values from learned outputs

Suppose a normalized feature has values $\hat{z} = (-1, 1)$, and the learned affine parameters are $\gamma = 3$, $\beta = 2$.

1. Calculate $y_i = \gamma \hat{z}_i + \beta$ for both values.
2. Find the mean and population variance of $y$.
3. Explain why the final outputs need not retain mean zero and variance one.

| Object | How obtained | Purpose |
|---|---|---|
| $\hat{z}$ | Computed normalization | Standardized relative values |
| $\gamma, \beta$ | Learned parameters | Task-dependent scale and shift |

# Answer: scale changes variance and shift changes mean

$$y = (3(-1) + 2,\ 3(1) + 2) = (-1, 5), \qquad \text{mean}(y) = 2, \quad \text{Var}(y) = 9.$$

- Both values are three units from the new mean; squared deviations are $9, 9$.
- For a zero-mean unit-variance feature, $\mathbb{E}[\gamma\hat{z} + \beta] = \beta$ and variance is $\gamma^2$.
- The exact normalization variance is approximate when $\epsilon$ is included; this exercise starts from the supplied values.
- Learned affine parameters restore flexibility without changing which statistics were used earlier.

# BatchNorm has different training and evaluation behavior

| Mode | Mean and variance | What stays learned |
|---|---|---|
| Training | Current mini-batch statistics; update running estimates | $\gamma$ and $\beta$ receive gradients |
| Evaluation | Saved running estimates from training | Same $\gamma$ and $\beta$; no new training statistics |

- A single test image should not need an artificial test batch to define its normalization.
- Calling `model.eval()` changes BatchNorm and Dropout behavior.
- `torch.no_grad()` prevents gradient recording; it does not switch layer behavior.

**Domain shift**

If new-camera activations differ strongly from training, saved running statistics can become inappropriate.

# Saved statistics define a single-case inference calculation

Let saved mean be 4, saved variance 5, and learned $\gamma = 2, \beta = 1$. For one new activation $z = 3$,

$$y = 2(3 - 4)/\sqrt{5 + \epsilon} + 1 \approx 0.106.$$

| Procedure | Statistics | Consequence |
|---|---|---|
| Evaluation mode | Saved training estimates | The single case has a fixed transform |
| Treat one case as its own batch | Mean 3; zero variance | Normalized value is zero; output becomes 1 |

**Meaning**

The second row is not the intended BatchNorm inference rule. Disabling gradients alone does not select the first row.

# LayerNorm normalizes features within one example

For a token vector $h = (h_1, \ldots, h_D)$, LayerNorm computes statistics over its $D$ feature coordinates.

| Property | BatchNorm | LayerNorm |
|---|---|---|
| Typical axes | Same channel across batch/spatial positions | Features within a token/example |
| Batch dependence | Yes, during training | No dependence on other examples |
| Evaluation statistics | Saved running estimates | Current token/example statistics |
| Common use | CNNs with suitable batches | Transformer hidden vectors |

### Two tokens

$(1, 3)$ normalizes to approximately $(-1, 1)$; $(10, 14)$ also does so, each with its own mean and scale.

### Choice

Read the tensor shape and normalization axes before choosing the layer.

# LayerNorm: normalize one token and apply feature-wise parameters

For $h = (1, 2, 3)$, mean is 2 and population variance is $2/3$.

$$\hat{h} = (-1, 0, 1)/\sqrt{2/3} \approx (-1.225, 0, 1.225).$$

Using $\gamma = (1, 1, 2)$ and $\beta = (0, 1, 0)$ gives $y \approx (-1.225, 1, 2.449)$.

- Statistics are computed across this token's three features.
- Learned affine parameters commonly have one value per normalized feature.
- No other token or batch partner was needed for this calculation.
- Applying different affine parameters after normalization need not preserve zero output mean.

# A batch partner changes BatchNorm but not this LayerNorm

Hold token $A = (1, 3)$ fixed. Compare batches $A, (3, 5), (5, 7)$ and $A, (3, 5), (9, 11)$.

| Normalization | First batch: output for $A$ | Second batch: output for $A$ |
|---|---|---|
| Feature-wise BatchNorm | $(-1.225, -1.225)$ | approximately $(-0.981, -0.981)$ |
| LayerNorm within $A$ | $(-1, 1)$ | $(-1, 1)$ |

Values omit affine parameters and tiny $\epsilon$.

### Read the axes

BatchNorm compares the same coordinate across cases. LayerNorm compares coordinates within $A$. A change to a partner affects only the former calculation here.

# A residual path gives a layer an identity option

$$h_{\ell+1} = h_\ell + F_\ell(h_\ell), \qquad \frac{\partial h_{\ell+1}}{\partial h_\ell} = I + \frac{\partial F_\ell}{\partial h_\ell}.$$

- A block learns a correction $F$ to the input rather than the entire mapping from scratch.
- If $F$ is near zero, the block approximately preserves the representation.
- The identity term creates a direct gradient path; it does not guarantee that every long product is stable.
- Addition requires matching shapes. A learned projection can match channel count or stride when needed.

## Scalar illustration

If $h = 3$ and $F(h) = -0.2$, the block outputs 2.8, a small learned correction.

# Backpropagate through one residual correction

Use a scalar correction $F(h) = 0.2h$ and residual output $y = h + F(h) = 1.2h$.

$$\frac{dy}{dh} = 1.2.$$

| Mapping | Local derivative | Input gradient if $dL/dy = 2$ |
|---|---|---|
| Ordinary $y = 0.2h$ | 0.2 | 0.4 |
| Residual $y = h + 0.2h$ | 1.2 | 2.4 |

- The identity path contributes 1 in the residual derivative.
- This example shows a direct gradient route, not universal improvement: other $F$ derivatives can amplify or cancel it.
- In vector layers, shapes must match before adding the paths.

# Regularization methods act at different locations

| Method | What it changes | What it encourages |
|---|---|---|
| Weight decay | Parameter update | Smaller parameter scale |
| Dropout | Training activations | Less reliance on a fixed feature combination |
| Data augmentation | Training examples | Suitable invariance to allowed transformations |
| Early stopping | Training duration/checkpoint | Stop before validation performance degrades |

- These mechanisms can complement one another, but adding all of them at maximum strength can prevent learning.
- A model with high training error may need better optimization, features or capacity before stronger regularization.

### Ask first

Where is the problem: fitting the training set, or generalizing beyond it?

# A training problem and a generalization problem need different first moves

| Illustrative observation | First hypothesis | Useful next check |
|---|---|---|
| Training 55%, validation 54% | Model has not learned enough | Tiny-subset fit; labels; optimization |
| Training 99%, validation 73% | Generalization or domain gap | Errors; split; suitable regularization |
| Training loss non-finite | Numerical or input error | Locate first invalid quantity |

- Initialization, normalization and residuals primarily help signal propagation and optimization.
- Decay, Dropout and augmentation target other fitting behaviors.
- Mechanisms can overlap; diagnoses are plausible hypotheses, not guaranteed causes from one number.

# Weight decay and L2 coincide for plain SGD

For a data loss $L(w)$ and L2 penalty $\frac{\lambda}{2}\|w\|^2$,

$$w_{t+1} = w_t - \eta(\nabla L(w_t) + \lambda w_t) = (1 - \eta\lambda)w_t - \eta\nabla L(w_t).$$

## One number

With $w = 2$, $g = 0.5$, $\eta = 0.1$, $\lambda = 0.01$, the next weight is $0.999(2) - 0.1(0.5) = 1.948$. Without decay it is 1.95.

- The shrinkage is proportional to the current parameter value.
- The strength depends on the loss reduction convention and learning rate.
- Biases and normalization parameters are often excluded from decay, depending on the model convention.

## Adaptive updates

Adding L2 to an Adam gradient is not generally identical to separately shrinking parameters.

# Repeated weight decay produces multiplicative shrinkage

Isolate decay by setting the data gradient to zero. Let $w_0 = 2$, $\eta = 0.1$, $\lambda = 0.01$.

$$w_{t+1} = 0.999 w_t, \qquad w_t = 2(0.999)^t.$$

| Update count | Weight | Interpretation |
|---|---|---|
| 1 | 1.998 | A small proportional reduction |
| 2 | 1.996002 | The next reduction acts on the new value |
| 10 | approximately 1.98009 | Many small shrinkages accumulate |

### Scope

Real training also has data gradients. The toy sequence isolates decay; it does not predict the complete parameter trajectory.

# AdamW decouples shrinkage from adaptive gradients

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

where $m_t$, $v_t$ use the data-loss gradient in this simplified description.

- Adam with L2 puts $\lambda w$ inside the gradient and hence inside its moments and adaptive scaling.
- AdamW applies a separate parameter shrinkage; $\lambda$ has a more direct decay interpretation.
- Weight decay is not a probability of deleting parameters and does not create sparse weights in the L1 sense.

### Our classifier

Tune a small set of decay strengths together with the learning rate. Keep the data split fixed and choose by validation performance.

### Practical question

Is the network fitting too little, fitting appropriately, or fitting noise?

# A coordinate divisor changes the effect of an L2 term

To isolate the distinction, freeze an illustrative adaptive divisor at $d = (1, 10)$ for one step. Take $w = (1, 1)$, data gradient zero, $\eta = 0.1$, $\lambda = 0.1$.

| Rule in this illustration | Change | New weights |
|---|---|---|
| Divide the L2 term by $d$ | $-\eta\lambda w/d = (-0.01, -0.001)$ | $(0.99, 0.999)$ |
| Separate weight shrinkage | $-\eta\lambda w = (-0.01, -0.01)$ | $(0.99, 0.99)$ |

**Scope**

This frozen-divisor calculation explains unequal shrinkage. A full Adam update also changes moments and needs their bias correction.

# Dropout changes activations only during training

For drop probability $p$ and independent mask $r_j \sim \text{Bernoulli}(1-p)$, inverted dropout uses

$$\tilde{h}_j = \frac{r_j h_j}{1-p} \quad \text{during training,} \qquad \tilde{h}_j = h_j \quad \text{during evaluation.}$$

- Different masks discourage excessive reliance on the same feature combination.
- The $1/(1-p)$ factor preserves the expected activation: $\mathbb{E}[\tilde{h}_j] = h_j$.
- Dropping an activation for one update does not permanently remove a neuron.
- Dropout placement and strength depend on architecture; it is not automatically helpful everywhere.

**Parameter names**

Check whether a library argument means drop probability or keep probability.

# Shared dropout masks preserve an average score, not an average loss

Let $h = (2, 4)$, a linear score $s = h_1 + h_2$, drop probability 0.5, and target score 6.

| Mask | Inverted-dropout score | Squared loss $\frac{1}{2}(s-6)^2$ |
|---|---|---|
| $(0,0)$ | 0 | 18 |
| $(1,0)$ | 4 | 2 |
| $(0,1)$ | 8 | 2 |
| $(1,1)$ | 12 | 18 |

The mean score is 6, but mean loss is $(18 + 2 + 2 + 18)/4 = 10$.

### Meaning

Preserving expected activations does not preserve expected nonlinear losses. The same model weights are trained under different masks.

# Dropout: one mask is noisy, its expectation is not

Take $h = (2, 4)$ and $p = 0.5$. Each coordinate is either dropped or doubled.

| Mask $r$ | Training output $\tilde{h}$ | Probability |
|---|---|---|
| $(0, 0)$ | $(0, 0)$ | $1/4$ |
| $(1, 0)$ | $(4, 0)$ | $1/4$ |
| $(0, 1)$ | $(0, 8)$ | $1/4$ |
| $(1, 1)$ | $(4, 8)$ | $1/4$ |

The average over masks is $(2, 4)$, equal to $h$. Evaluation uses $(2, 4)$ directly.

### What this does not claim

A nonlinear network's full prediction is not exactly the average prediction of every masked subnetwork. The ensemble analogy is useful but approximate.

# Exercise: explain repeated predictions under different mode switches

Use the same linear score and input $(2, 4)$.

| Setting | Possible scores | What the switch controls |
|---|---|---|
| `train()` with `no_grad()` | 0,4,8,12 under different masks | No gradient record; Dropout still active |
| `eval()` with gradient recording | 6 repeatedly | Dropout off; derivatives may still be recorded |

- Explain why the first setting can give inconsistent validation predictions.
- State the usual combination for fixed validation: evaluation mode plus disabled gradient recording.
- Do not confuse a layer-behavior switch with a parameter-freezing decision.

# Training mode is a layer behavior, not a gradient switch

| Operation | Training | Validation / inference |
|---|---|---|
| BatchNorm | Batch statistics; running-state updates | Saved running statistics |
| Dropout | Random masks and rescaling | All activations available |
| Gradient recording | Required for backward | Usually disabled |
| Parameter update | Optimizer step | None |

- `model.train()` and `model.eval()` select behaviors.
- `no_grad()` or `inference_mode()` controls recording.
- Freezing parameters is a third choice: a frozen backbone can still have mutable BatchNorm state if left in training mode.

### Connect the ideas

An unchanged weight tensor does not necessarily imply unchanged model outputs.

# Read training and validation curves together

| Observation | Plausible explanation | First useful check |
|---|---|---|
| Both losses high | Optimization, weak representation or insufficient capacity | Data / loss correctness; tiny-subset fitting |
| Training low, validation high | Overfitting, shift or leakage in the comparison | Split quality; regularization; more relevant data |
| Both improve slowly | Small rate, poor scaling or difficult task | Rate range; initialization; gradient flow |
| Loss becomes non-finite | Invalid values or unstable operations | Locate first NaN/Inf; then inspect gradients/rate |

**A curve is evidence**

It narrows the possibilities. It does not uniquely identify the cause from shape alone.

# Classify three curve patterns before proposing a change

For this example, use balanced binary cross entropy. A constant probability 0.5 has loss $\log 2 \approx 0.693$.

| Illustrative pattern | What it suggests | Justified first action |
|---|---|---|
| Train 0.90, valid 0.92 | Weak fitting on both sets | Check learning on a small clean subset |
| Train 0.05, valid 0.90 | Large generalization gap | Inspect errors, split and regularization |
| Train becomes NaN | Invalid arithmetic or values | Locate the first non-finite tensor |

- These numbers alone do not uniquely reveal the cause.
- Explain why “increase Dropout” is a poor universal answer.
- A useful hypothesis predicts which observation should change after the intervention.

# A learning-curve example leads to a concrete decision

| Epoch | 1 | 5 | 10 | 20 |
|---|---|---|---|---|
| Training cross-entropy | 1.10 | 0.55 | 0.22 | 0.08 |
| Validation cross-entropy | 1.15 | 0.64 | 0.61 | 0.82 |
| Validation accuracy | 55% | 73% | 76% | 75% |

- The model keeps fitting training cases, but validation loss worsens after epoch 10.
- Accuracy changes little; probabilities can still become more confidently wrong.
- If validation loss is the declared selection criterion, save the epoch-10 checkpoint.
- Try one justified change, such as moderate augmentation or decay, then rerun the same comparison.

**Test data**

Do not choose the epoch or intervention using the final test set.

# Equal accuracy can hide very different cross-entropy

Two cases have binary labels $(1, 0)$. Compare predicted probabilities for class 1.

| Model | Class-1 probabilities | Accuracy | Mean cross-entropy |
|---|---|---|---|
| A | $(0.6, 0.6)$ | 50% | $[-\log 0.6 - \log 0.4]/2 \approx 0.714$ |
| B | $(0.9, 0.99)$ | 50% | $[-\log 0.9 - \log 0.01]/2 \approx 2.355$ |

- Both predict class 1 twice, so both get one case right.
- Model B is much more confidently wrong on the second case.
- Cross-entropy evaluates the probability of the true class, not only the argmax label.

# Early stopping selects a checkpoint, not a magic epoch

1. Declare a validation metric and whether lower or higher is better.
2. Evaluate at a fixed interval; save the best checkpoint so far.
3. Allow a patience window for noisy improvements before stopping.
4. Restore the best checkpoint, which can be earlier than the stopping epoch.

### Patience of three evaluations

Best loss at epoch 10; no sufficient improvement at 11,12,13. Stop after 13, restore epoch 10.

- Compare validation with inference behavior and fixed preprocessing.
- Use a separate test set for the final performance estimate.

### Link to regularization

The selected training horizon limits how far the model fits idiosyncrasies of the training set.

# Exercise: find the best checkpoint under a patience rule

Validation losses are $1.00, 0.70, 0.65, 0.66, 0.64, 0.67, 0.68$ at evaluations 1–7. Patience is two consecutive evaluations without improvement.

| Evaluation | Loss | Decision to explain |
|---|---|---|
| 3 | 0.65 | New best |
| 4 | 0.66 | One non-improvement |
| 5 | 0.64 | New best; counter resets |
| 6 and 7 | 0.67,0.68 | Two non-improvements |

**Answer**

Stop after evaluation 7 and restore checkpoint 5. The stopping time and selected checkpoint differ. State whether patience counts evaluations or epochs.

# Fit a tiny subset before changing the architecture

1. Pick 16–32 correctly labeled training examples and inspect them manually.
2. Temporarily simplify augmentation and regularization.
3. Train repeatedly on this subset; the network should substantially reduce its loss.
4. If it cannot, inspect labels, tensor shapes, logits/loss compatibility, learning rate and whether parameters receive gradients.

### Classification check

Cross-entropy expects class logits and compatible integer labels; applying an incompatible extra softmax or label encoding can break the intended loss.

### Purpose

A successful tiny-subset fit checks that learning is possible. It does not demonstrate generalization or justify the final architecture.

# A tiny-subset failure points to specific checks

| Observation on 32 clean cases | Candidate issue | A direct check |
|---|---|---|
| Loss never changes | Frozen parameters or no update | Check trainable parameters and their changes |
| Loss changes but labels seem swapped | Target encoding mismatch | Inspect example, label and class order |
| All outputs identical | Input/shape or initialization issue | Inspect model inputs and gradients |

- Temporarily simplify heavy regularization and random transforms.
- The test intentionally probes fitting ability, not independent generalization.
- Fix a concrete failure before spending resources on a larger architecture search.

# Augmentation teaches an allowed invariance

We train on transformed examples $(T(x), y)$ only when the transformation preserves the task label.

- For some object classification tasks: small translation, crop, brightness changes or horizontal flips can be reasonable.
- For digit recognition: a rotation can change identity; a flip can make an invalid character.
- For defect detection: cropping out a tiny defect while keeping a positive label creates a contradictory training example.
- For medical or scientific images: orientation, intensity and scale may carry physical meaning.

### Task-first choice

Specify what should stay constant before selecting transformations. Augmentation expands variation; it does not create new independent labeled subjects.

# A crop needs the evidence required by the label

In a $100 \times 100$ image, a crack lies in $x \in [30, 35]$, $y \in [40, 45]$.

| Crop region | Crack included? | Label interpretation |
|---|---|---|
| $[0, 20] \times [0, 20]$ | No | A positive crop label is unsupported |
| $[20, 50] \times [30, 60]$ | Yes | Positive evidence remains visible |

- A transform can preserve file identity while removing the task's evidence.
- For detection or segmentation, move spatial annotations with the image.
- For a classification label, constrain crops or avoid those that contradict the intended target.

# A transformation can preserve appearance but change the label

| Transformation | Potential benefit | Label check |
|---|---|---|
| Small translation | Less dependence on object position | Entire target remains visible |
| Moderate brightness | Robustness to lighting variation | Defect remains detectable |
| Random crop | Less dependence on background | Positive crop still contains the defect |
| Horizontal flip | Orientation robustness | Left/right has no label meaning |

**Counterexample**

A positive image contains a crack in the lower-right corner. A top-left crop contains no crack. Reusing the positive label teaches the wrong relationship.

**Repair**

Constrain the crop, update a spatial label, or avoid the transformation for this task.

# Exercise: accept a transform only with a task-specific reason

| Transformation | Task | Judgment and reason |
|---|---|---|
| Small translation, object visible | Object classification | Usually plausible position invariance |
| Crop removes the only crack | Defect classification | Reject the unchanged positive crop label |
| Left/right flip | Laterality classification | Reject if left/right is the target |
| Moderate lighting variation | Robust object recognition | Plausible only if evidence remains usable |

**Discuss**

Name one situation where a commonly used image transform is wrong. Then describe what should remain unchanged for your own task.

# Separate stochastic training transforms from fixed evaluation

- Split original cases before creating augmented variants; all variants of one source stay in the same split.
- Training can use random transformations. Validation should use fixed, task-appropriate preprocessing for comparable curves.
- Compute normalization statistics on training data, not on the combined training and validation sets.
- Keep labels and spatial annotations synchronized with transforms.

### One image, two transforms

Training may take a random crop that preserves its label. Validation uses a defined resize/crop rule. Both use the same intended input scale.

### Optional test-time augmentation

It is an explicit prediction procedure with extra computation, not an accidental leftover training transform.

# Augmented twins can leak across a random row split

Suppose 100 original images each produce 10 related variants.

| Split procedure | Counts | Risk |
|---|---|---|
| Split 100 originals first | 80 train sources, 20 validation sources | Source groups remain separate |
| Randomly split 1000 variant rows | About 800 / 200 rows | Twins of one original often cross the boundary |

- Under independent 80/20 row allocation, one 10-variant group appears on both sides with probability $1 - 0.8^{10} - 0.2^{10} \approx 0.893$.
- Related variants are not independent new source images.
- Evaluate generalization to held-out sources, subjects or acquisition units required by the task.

# Transfer learning reuses representations, not old labels

Let a pretrained backbone produce features $h_\phi(x)$ and a new head predict

$$p(y \mid x) = \text{softmax}(Wh_\phi(x) + b).$$

- The backbone learned visual features from a source task; the new head matches the target label set.
- A frozen backbone is a feature extractor. A linear head learns target class boundaries with fewer trainable parameters.
- Fine-tuning updates some or all backbone layers using target data.
- Source and target similarity, target data size and available computation guide how much to adapt.

**Our task**

Replace a general image classifier's output layer with a two-class defect head. Reusing the old 1000-class output labels would not solve the new task.

# Frozen features connect transfer to a linear classifier

Take a frozen feature $h = (1, 2)$, two output rows $W_0 = (1, 0)$, $W_1 = (0, 1)$, and zero biases.

$$o = (1, 2), \qquad p = \mathrm{softmax}(o) \approx (0.269, 0.731).$$

For target class 1, loss is $-\log 0.731 \approx 0.313$.

- The new head maps source features to target labels.
- Freezing $h$ leaves the classifier's weights trainable; training is still supervised learning.
- The head's two rows have a declared class order. Target label 1 refers to the second row.
- Useful source features are a starting representation, not a guarantee that a linear boundary is sufficient.

# A staged transfer procedure makes adaptation understandable

1. Match the pretrained model's input format, scale and channel convention.
2. Replace the task head and train it while the backbone is frozen.
3. Examine validation errors; unfreeze a small number of upper layers if adaptation is needed.
4. Use a lower learning rate for pretrained layers than for a newly initialized head when appropriate.
5. Decide explicitly whether BatchNorm running statistics are fixed or adapted.

**Starting experiment**

Head rate $10^{-3}$; pretrained layers $10^{-4}$. These are trial settings to validate, not universal constants.

**Do not damage useful features**

A large uniform rate can overwrite representations before the new head becomes useful.

# Trainable parameters and mutable states are different objects

| Object in a frozen-backbone run | Optimizer update? | Other behavior to choose |
|---|---|---|
| Backbone weight parameters | No, if deliberately frozen | Forward computation still used |
| New head parameters | Yes | Match target labels and loss |
| BatchNorm running estimates | Not ordinary gradient parameters | Can change in training mode |
| Dropout mask | Not a saved learned weight | Random in training, inactive in evaluation |

### A concrete check

Inspect which parameters the optimizer receives and separately decide module modes. Freezing weights does not automatically freeze every form of model state.

# Transfer example: count parameters and choose a first test

Suppose a backbone outputs a 512-dimensional vector and has 10 million parameters. A two-class linear head has

$$2 \times 512 + 2 = 1026 \quad \text{trainable parameters.}$$

| Strategy | Updated parameters | Main tradeoff |
|---|---|---|
| Frozen backbone | 1026 | Efficient; representation stays fixed |
| Upper-layer tuning | Head plus selected layers | Some adaptation; moderate data demand |
| Full tuning | About 10 million plus head | Flexible; larger overfitting/compute burden |

### With 400 labeled images

Begin with the frozen-head baseline, then compare selective tuning under the same split. Parameter count alone does not prove which option wins.

# Calculate a small pretrained-layer step and a larger head step

For an illustrative SGD update, use the same scalar gradient $g = 0.5$ in two parameter groups.

| Group | Initial value | Rate | New value |
|---|---|---|---|
| Pretrained layer | 2 | $10^{-4}$ | $2 - 0.00005 = 1.99995$ |
| New head | 0.2 | $10^{-3}$ | $0.2 - 0.0005 = 0.1995$ |

- The newly initialized head receives a ten-times larger change for the same supplied gradient.
- Real gradients differ by layer; this example isolates the learning-rate factor.
- Pretrained features may need cautious adaptation, while the new label mapping must develop from its own start.

# Data size and domain difference guide how much to tune

| Target situation | Sensible starting strategy | What to examine |
|---|---|---|
| Small data, similar domain | Frozen features / new head | Whether errors need representation changes |
| More data, similar domain | Tune upper layers or more layers | Benefit relative to frozen baseline |
| More data, different domain | Broader tuning may be needed | Input differences and task evidence |
| Small data, different domain | Conservative tuning; gather relevant labels | Avoid confident promises from little evidence |

### A rule of thumb

The matrix selects a starting experiment. Target validation decides; there is no architecture-independent guarantee.

### Domain-specific inputs

Medical scans may require changes to input channels and preprocessing in addition to the classification head.

# Choose a transfer baseline for four target datasets

| Illustrative target data | Starting experiment | Reason |
|---|---|---|
| 100 natural images, similar labels | Frozen features plus new head | Few labels; source features may transfer |
| 10000 related natural images | Tune upper layers | More evidence for adaptation |
| 200 scientific images, different acquisition | Conservative transfer and gather labels | Large gap with limited supervision |
| 10000 scientific images | Compare broader tuning | More data can support changes |

**Boundary**

Counts are teaching scenarios, not universal thresholds. Target validation and actual similarity decide whether the starting strategy is adequate.

# A minimal training loop puts the ideas into code

```
model.train()
for x, y in train_loader:
    optimizer.zero_grad(set_to_none=True)
    logits = model(x)
    loss = criterion(logits, y)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
# scheduler.step() goes at its documented update/epoch boundary
```

- Clear previous gradients; compute the current loss and gradients; then update.
- Gradient clipping is optional and limits large gradients; it cannot repair wrong labels or restore vanished gradients.
- Use the scheduler at the interval its definition expects.

# Exercise: locate errors in a short training and validation sketch

```
for x, y in train_loader:
    loss = criterion(model(x), y)
    loss.backward()
    optimizer.step()
model.train()
with torch.no_grad():
    scores = model(valid_x)
```

- The loop is intended to make independent ordinary mini-batch updates. Which missing step changes that intention?
- Which mode call is inappropriate for fixed validation with BatchNorm or Dropout?
- Explain why `no_grad()` does not repair the mode error.

**Task**

Explain the mathematical or layer behavior behind each correction, rather than merely naming a missing line.

# Validation measures the model without training it

```
model.eval()
total_loss, correct, count = 0.0, 0, 0
with torch.no_grad():
    for x, y in valid_loader:
        logits = model(x)
        loss = criterion(logits, y)  # mean over this batch
        total_loss += loss.item() * y.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        count += y.size(0)
valid_loss, valid_acc = total_loss / count, correct / count
```

### Why multiply by batch size?

The final batch can be smaller. Weight batch means by case count to obtain the correct dataset mean. No backward or optimizer step belongs here.

# Answer: reset, evaluation behavior and weighted validation means

| Issue | Correction | Reason |
|---|---|---|
| Ordinary update keeps old gradients | Clear gradients before the new backward | Do not unintentionally accumulate batches |
| Validation uses training mode | Call evaluation mode | Fix BatchNorm/Dropout behavior |
| Batch losses averaged equally | Weight by batch size | Estimate the per-case dataset mean |

### Two unequal batches

Means 0.2 for 96 cases and 1.0 for 4 cases give $(96(0.2) + 4(1.0))/100 = 0.232$, not $(0.2 + 1.0)/2 = 0.6$.

# Case discussion: choose the next intervention

A 400-image defect classifier has 99% training accuracy and 73% validation accuracy. The validation split contains parts from machines not seen during training.

1. Confirm the split and labels reflect the intended new-machine task.
2. Inspect mistakes: background dependence, lighting changes, or missing small defects?
3. Choose a relevant change: label-preserving lighting augmentation, suitable decay, or selective transfer tuning.
4. Compare the same validation metric with the same split and budget.

### An unjustified response

Increasing network size solely because validation accuracy is low may worsen memorization.

### Discuss

What observation would make you suspect preprocessing rather than overfitting?

# End-of-lecture exercise: connect an error pattern to a mechanism

A small defect classifier fits training images well but fails mainly on a new machine's darker images. The crack is still visible in those images.

1. Check that preprocessing and labels match the declared target task.
2. Propose moderate, label-preserving lighting augmentation and compare against the fixed baseline.
3. Explain why simply increasing hidden-layer width does not directly target this observed shift.
4. Decide which validation examples and metric should reveal whether the change helps.

## Answer structure

Observation $\rightarrow$ plausible cause $\rightarrow$ matched mechanism $\rightarrow$ evidence. Keep the new-machine split intact rather than hiding the difficulty by mixing sources.

# Connect each method to the object it changes

| Object | Method | Explain it in one sentence |
|---|---|---|
| Gradient direction | Momentum | Remember consistent past directions |
| Coordinate scale | Adam | Divide by a history-based gradient scale |
| Global step length | Learning-rate schedule | Change how far updates move over time |
| Activation scale/path | Initialization, normalization, residuals | Keep information usable across layers |
| Parameter/feature reliance | Decay, Dropout | Restrict fitting to unstable details |
| Training examples | Augmentation | Teach task-valid transformations |
| Representation | Transfer learning | Reuse and adapt source features |

**Use the chain**

First make learning work, then improve generalization, then compare target-task performance.

# Exit questions and the bridge to sequence models

1. With momentum, why do gradients $2, 2$ and $2, -2$ produce different second moves?
2. What changes between BatchNorm training and evaluation? How is LayerNorm different?
3. Why can a positive defect image become incorrectly labeled after a crop?
4. For a small target dataset, what baseline should precede full fine-tuning?

### A complete explanation

State the mechanism, use a small calculation or concrete example, and connect it to a training decision.

### Next lecture

Sequence models add a new question: how should information move across positions? RNNs use recurrent state; attention retrieves from visible positions.